

Docker Containerization Project Report (Rewritten Version)

March 2026

Name: Keshav Kumar
SAP ID: 500125136
Roll No: R2142231973

1. Overview

This report explains the development of a Docker-based multi-container system. It highlights architecture design, performance improvements, networking setup, and storage handling techniques for a backend service and database.

2. Build Optimization Techniques

Efficient Docker builds help reduce size, improve speed, and enhance security. Several optimization strategies were applied:

2.1 Multi-Stage Build Strategy

Separate build and runtime environments were used. Dependencies are installed and compiled in one stage, while only final output files are transferred to a smaller runtime image.

2.2 Docker Layer Caching

By structuring Dockerfile steps efficiently, repeated builds avoid reinstalling dependencies unless required, significantly improving rebuild speed.

2.3 Lightweight Base Images

Minimal images like Alpine Linux were used to reduce overall image size and limit security vulnerabilities.

2.4 Ignoring Unnecessary Files

A `.dockerignore` file excludes files such as logs, git data, and `node_modules` to optimize build context.

3. Image Size Impact

Component	Base Image	Before Optimization	After Optimization
Backend	node:18	~900 MB	~160 MB
Database	postgres:15	~400 MB	~250 MB
Total	-	~1.3 GB	~410 MB

Overall, optimization reduced the size by around 68%, improving efficiency.

4. Networking Approach

A custom bridge network allows containers to communicate securely without exposing internal services externally.

4.1 Network Configuration

The backend and database containers are connected through a dedicated network, enabling communication using service names instead of fixed IPs.

4.2 Network Creation

A custom subnet was created using Docker commands, ensuring proper isolation and connectivity between services.

5. macvlan vs ipvlan

Both networking drivers allow containers to behave like physical devices, but differ in performance and complexity.

macvlan assigns unique MAC addresses, while ipvlan shares host MAC addresses and reduces network overhead.

5.1 Preferred Approach

For this project, a bridge network was sufficient. However, ipvlan is generally preferred in production due to better performance and simpler setup.

6. Data Persistence

Docker volumes were used to ensure database data remains intact even after container restarts or deletions.

7. Summary

The project demonstrates efficient containerization using Docker, including reduced image sizes, secure networking, and reliable data persistence.